

LegoNet: A Fast and Exact Unlearning Architecture

Sihao Yu, Fei Sun, Jiafeng Guo*, Ruqing Zhang, Xueqi Cheng

University of Chinese Academy of Sciences, Beijing, China

CAS Key Lab of Network Data Science and Technology, Institute of Computing Technology,
Chinese Academy of Sciences, Beijing, China

{yusihao, sunfei, guojiafeng, zhangruqing, cxq}@ict.ac.cn

Abstract

Machine unlearning aims to erase the impact of specific training samples upon deleted requests from a trained model. Re-training the model on the retained data after deletion is an effective but not efficient way due to the huge number of model parameters and re-training samples. To speed up, a natural way is to reduce such parameters and samples. However, such a strategy typically leads to a loss in model performance, which poses the challenge that increasing the unlearning efficiency while maintaining acceptable performance. In this paper, we present a novel network, namely *LegoNet*, which adopts the framework of “fixed encoder + multiple adapters”. We fix the encoder (i.e., the backbone for representation learning) of *LegoNet* to reduce the parameters that need to be re-trained during unlearning. Since the encoder occupies a major part of the model parameters, the unlearning efficiency is significantly improved. However, fixing the encoder empirically leads to a significant performance drop. To compensate for the performance loss, we adopt the ensemble of multiple adapters, which are independent sub-models adopted to infer the prediction by the encoding (i.e., the output of the encoder). Furthermore, we design an activation mechanism for the adapters to further trade off unlearning efficiency against model performance. This mechanism guarantees that each sample can only impact very few adapters, so during unlearning, parameters and samples that need to be re-trained are both reduced. The empirical experiments verify that *LegoNet* accomplishes fast and exact unlearning while maintaining acceptable performance, synthetically outperforming unlearning baselines.

Introduction

While machine learning brings great benefits, it also brings critical privacy concerns in recent years due to the unintended memorization of the training data which might contain users’ sensitive information (Carlini et al. 2019), e.g., users’ behavior data used to train recommender systems (Zhang et al. 2021) or personal information in the corpus for large natural language models (Carlini et al. 2021; Zanella-Béguelin et al. 2020). To protect the individual’s privacy, multiple data protection regulations, e.g., GDPR in the European Union, have enacted the “*right to be forgotten*” which requires the service providers that process personal data to delete user

data upon request (Alessandro and Mantelero 2013). However, only deleting data is not enough, considering that users’ information could be leaked from the trained models (Yeom et al. 2018; Carlini et al. 2019; Shokri et al. 2017; Zhang et al. 2021; Truex et al. 2019). To address this issue, machine unlearning has been proposed to erase the impact of specific training samples from a trained model upon deleted (i.e., unlearned) requests.

Numerous unlearning methods try to eliminate the impact of the unlearned samples by continually optimizing (fine-tuning) the trained model’s parameters (Guo et al. 2020; Gollatkar, Achille, and Soatto 2020a; Izzo et al. 2021). However, such methods typically can not guarantee that the impact from unlearned samples is completely removed, especially for the neural models. To protect privacy, exact unlearning is required. A naive way is to re-train the model from scratch using the retained training data. However, such a fully re-training strategy has a critical issue in efficiency since all model parameters should be re-trained on all re-trained samples once a sample is required to be unlearned (Bourtoule et al. 2021; Chen et al. 2022, 2021). Previous works have tried to reduce the samples that participate in re-training to accelerate the unlearning process (Bourtoule et al. 2021; Chen et al. 2022, 2021). As a representative, SISA splits the data into multiple disjoint shards for training multiple independent sub-models one to one (each sub-model adopts the same network as the original model unlearned by naive re-training). When unlearning a sample, it only needs to re-train the corresponding sub-model on the retained samples in the shard to which the unlearned sample belongs. As the number of shards increases, the samples involved in re-training are reduced, but the performance empirically drops significantly. To maintain acceptable performance, the degree to which data can be sharded is severely limited. Therefore, the SISA-based methods are still not efficient enough, especially when the amount of training data is large.

To further improve the unlearning efficiency, we propose a new idea that aims to reduce the number of parameters for re-training. Inspired by continual learning (Wang et al. 2021; Aljundi, Chakravarty, and Tuytelaars 2017), a feasible solution is to segment the parameters and reduce the impact region of the samples on the parameters. Generally, a neural model has a large backbone to learn representations, and then adopts a simple network, such as a linear layer, to

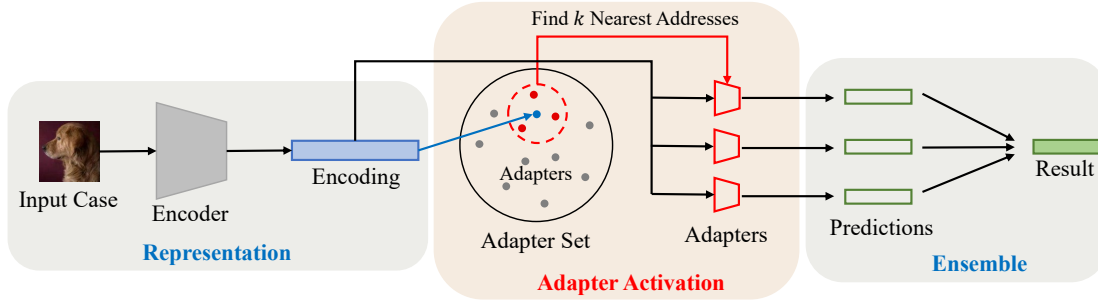


Figure 1: Overview of LegoNet. The feedforward computation of LegoNet has three main processes, i.e., representation, adapter selection, and ensemble. The input case is encoded into the encoding at first. Then, according to the encoding, k ($k = 3$ in the picture) adapters are selected from the adapter set. Each adapter has independent parameters and makes one candidate prediction when receiving the input encoding. At last, the candidate predictions of activated adapters are aggregated to the final result.

make predictions based on the representations. While the representation learning of the backbone does not necessarily require the use of the task’s training data. It indicates that the backbone can get rid of re-training. Then, the unlearning efficiency will be significantly improved.

Follow this idea, we present a novel neural network, named *LegoNet*, as shown in Fig. 1. Specifically, LegoNet consists of a fixed encoder (i.e., backbone) for representation and n isolated adapters for determining the predictions. Here, the encoder has been pre-trained on external data (e.g., a large generic dataset such as ImageNet (Russakovsky et al. 2015)), which ensures the model has enough ability for representation. Since the fixed encoder occupies most parameters of LegoNet, the re-trained parameters during unlearning are significantly reduced. However, empirically, the fixed encoder also leads to a loss in model performance. Here, LegoNet adopts the ensemble of multiple adapters to largely recover this performance loss.

Moreover, to further trade off model performance against unlearning efficiency, we design the adapter activation mechanism. As the encoder maps the input data from the sample space to the encoding space, we assign preset keys for adapters to represent their address in the encoding space. By this approach, we build the distance property between the adapters and the samples. For each sample, k ($k \ll n$) nearest adapters will be activated and adopted to train or infer. The ensemble of k activated adapters ensures acceptable performance. Moreover, the impact of any training sample is controlled in k definite adapters. Therefore, when unlearning a specific sample, LegoNet only needs to re-train k impacted adapters to achieve exact unlearning. Overall, such a mechanism further speeds up re-training in two ways. On the one hand, although the number of adapters’ parameters is already small, we further reduce the number of impacted adapters that need to be re-trained. On the other hand, the number of samples that should be adopted to re-train the impacted adapters is reduced, since the adapter can only be impacted by the nearby training samples in the encoding space. Since the number of both parameters and samples for re-training is effectively reduced, the unlearning speed of LegoNet is faster than that of SISA-based methods which only reduces the number of re-training samples.

To verify the exact performance of our unlearning ap-

proach, we conduct image classification experiments on CIFAR-10/100 (Torrabla, Fergus, and Freeman 2008). Experiments present that our model efficiently accomplishes exact unlearning and keeps the performance on the test set. Moreover, we compare LegoNet with SISA through ablation experiments to analyze the effects and advantages of our differences with SISA. Finally, we conduct extensive experiments with different hyperparameters for LegoNet to further analyze how our approach trades off model performance against unlearning speed.

Related Works

Machine unlearning (Bourtoule et al. 2021; Neel, Roth, and Sharifi-Malvajerdi 2021; Gupta et al. 2021) refers to a process that aims to remove the impact of the deleted samples of training data from a trained model. Such a process is also named as *decremental learning* (Cauwenberghs and Poggio 2001; Karasuyama and Takeuchi 2009), *selective forgetting* (Golatkar, Achille, and Soatto 2020a) or *data removal/deletion* (Guo et al. 2020; Ginart et al. 2019) in prior works. From the certainty of removal, existing methods of machine unlearning can be divided into exact unlearning and approximate unlearning.

Exact unlearning ensures that the unlearned data is completely deleted from the trained model. Previous studies typically achieve the exact data deletion at a cheaper computing cost than fully re-training for simple models or under specific conditions (Cauwenberghs and Poggio 2001; Cao and Yang 2015; Schelter 2020). For example, leave-one-out cross-validation speed up the exact unlearning for SVMs (Support Vector Machines) (Cauwenberghs and Poggio 2001; Karasuyama and Takeuchi 2009), provably efficient data deletion is designed for k -means clustering (Ginart et al. 2019), and fast data deletion for Naïve Bayes is achieved with the assumption of statistical query learning (training data is in a decided order) (Cao and Yang 2015). More generally, SISA (Sharded, Isolated, Sliced, and Aggregated) (Bourtoule et al. 2021) proposes a representative framework that supports partial re-training, based on the idea of “divide and conquer”. Specifically, its main steps include: (1) divide the training data into some disjoint shards; (2) copy multiple models and train them on different shards independently; (3) aggregate the results of all models for the final inference. Then, SISA

exactly re-trains the impacted sub-model, whose shard contains the removal data, for unlearning. Compared with fully re-training a model, re-training each model on tiny shards greatly improves the efficiency. Following this idea, an improved sharding algorithm is presented for the unlearning of the graph (Chen et al. 2021). The core of SISA is data segmentation, which reduces the amount of data received by each sharded model for re-training. However, the number of parameters for each re-training is still huge, which severely affects the efficiency. Only reducing the amount of data has limited potential for acceleration, when SISA increases the number of data shards, the performance typically drops significantly. In this paper, we try to reduce both parameters and samples for re-training to further speed up the unlearning.

Approximate unlearning relaxes the requirement for exact deletion (Guo et al. 2020; Golatkar, Achille, and Soatto 2020a; Izzo et al. 2021), i.e., it only accomplishes that the removed data is statistically unlearned with the guarantee that the unlearned model cannot be distinguished from an exact deletion model (Guo et al. 2020). Such approximate methods typically adopt gradient-based strategies to remove the influence of deleted samples from the trained model (Neel, Roth, and Sharifi-Malvajerdi 2021). Following the idea of gradient update, different Newton’s methods are proposed to approximate re-training for convex models, e.g., linear regression, logistic regression, and the last fully connected layer of a neural network (Guo et al. 2020; Golatkar, Achille, and Soatto 2020a,b). Since the core is to eliminate the influence of specified samples from the trained model, the gradient updates also can be decided according to the influence functions (Izzo et al. 2021). Compared with exact unlearning, approximate unlearning methods are usually more efficient. However, such gradient-based methods are limited by the condition of “convex”, thus they are hard to apply to non-convex models like deep neural networks. Moreover, such methods can not guarantee that the impacts of deleted data are completely removed. Thus, their security is insufficient from a privacy protection point of view.

LegoNet

In this section, we introduce our network, i.e., *LegoNet*, and the corresponding unlearning process. First, we introduced the network architecture and the feedforward calculation of LegoNet. Next, we introduce the training process and the unlearning process of LegoNet. Finally, we compare LegoNet with SISA to further present and analyze the advantages of our method.

Architecture and Feedforward Calculation

LegoNet is composed of a fixed encoder and n isolated adapters. The encoder, as the backbone of LegoNet, is a feature extractor adopted for representing the input samples. Specifically, it maps the samples to the encoding space. Here, the choice of the encoder’s architecture is relatively free. To deal with different kinds of inputs, we can adopt different architectures, such as BERT (Vaswani et al. 2017) for text tasks or ResNet (He et al. 2016) for image tasks. Each adapter contains a preset key to represent its address in the encoding

Symbol	Definition
x	The data instance.
y	The label of the data instance.
e	The encoding of the data instance.
$\mathcal{D}$	The training dataset.
En	The encoder of LegoNet.
a_j	The j^{th} adapter of LegoNet.
$\mathbf{K}_j$	The key of a_j .
$\mathbf{M}_j$	The sub-model of a_j .
θ_j	The parameters of $\mathbf{M}_j$.
$\mathcal{D}_{a_j}$	The set of samples that can activate a_j .

Table 1: Definitions of Main Symbols.

space, and an independent sub-model adopted for prediction based on the output of the encoder. Typically, the sub-model of each adapter only requires a simple architecture, such as a linear layer. To facilitate the following description, we adopt symbols for formulation. Table 1 gives the definitions of the main symbols used in this paper.

For inferring an input case x , the encoder firstly encodes x to the encoding as $e = \text{En}(x)$. This encoding determines the activation of the adapter. Specifically, for $j = 1$ to n , we can calculate the distance from the sample x to the adapter a_j as:

$$\delta(x, a_j) = \|\text{En}(x) - \mathbf{K}_j\| = \|e - \mathbf{K}_j\|, \quad (1)$$

where $\|\cdot\|$ denotes the norm (e.g., L_2 -norm is adopted in our experiments). Then, k ($1 \leq k \ll n$) nearest adapters will be activated and selected. Assume that the activated adapters are $a'_1, a'_2, \dots, a'_k$. According to the encoding and the activated adapters, the final prediction for x is calculated through ensemble as:

$$p = \frac{1}{k} \sum_{z=1}^k \mathbf{M}'_z(e, \theta'_z), \quad (2)$$

where $\mathbf{M}'_z$ denotes the sub-model of a'_z , and θ'_z denotes the parameters of $\mathbf{M}'_z$.

Pre-Settings and Training

In this section, we introduce: (1) the pre-settings for the encoder and the keys of the adapters; (2) the training process for the sub-model of each adapter. Assume the training dataset has N samples, specifically, $\mathcal{D} := \{(x_i, y_i) : 1 \leq i \leq N\}$, where the subscript i represents the index of the instance.

For the encoder, before being adopted in LegoNet, it should be pre-trained on external data to ensure its enough ability for representation. The choice for the external data can be public free data, desensitized data, or non-personal data. This setting ensures that the data will not be required to unlearn samples. In this way, LegoNet does not need to reprocess the encoder. In particular, data from a similar domain to the target task is a better choice for pre-training the encoder, which can effectively improve the performance of LegoNet.

For the keys of the adapters, after the encoder is pre-trained, the keys ($\mathbf{K}_1, \dots, \mathbf{K}_n$) should be initialized to have the similar distribution with the encodings of training samples in $\mathcal{D}$. In the specific implementation of initialization, we ensure that the key distribution of adapters and the encoding

distribution of training samples are the same on expectation via Monte Carlo Sampling (Shapiro 2003). Specifically, we uniformly sample n instances (denoted as $x_1^*, \dots, x_n^*$) from $\mathcal{D}$ and adopt the encoding of x_j^* to build the key of a_j as:

$$\mathbf{K}_j = \text{En}(x_j^*) + \xi_j, \quad (3)$$

where ξ_j represents a slight random perturbation. ξ_j is adopted to guarantee that the information of the sample is not directly recorded in LegoNet. After initialization, the keys of adapters should be fixed.

The keys directly determine which samples the adapters will process (including training and inference). During training, when each sample activates its k nearest adapters, we record the set of samples for activating different adapters. Specifically, we denote the set of samples that activate a_j as $\mathcal{D}_{a_j}$. Our setting for keys make $\{|\mathcal{D}_{a_j}|\}_{j=1, \dots, n}$ roughly follow the uniform distribution. Since there are more adapters where the samples are dense in the encoding space (similarly, there are fewer adapters where the samples are sparse). The balanced samples for each adapter are beneficial to the training quality of the sub-models, because it mitigates the situation that a sub-model is lack of enough training data. Moreover, it also facilitates the subsequent unlearning task by avoiding that a sub-model should be re-trained by a large number of samples. In particular, for the effect of the distribution of $\{|\mathcal{D}_{a_j}|\}_{j=1, \dots, n}$ on unlearning efficiency, we provide a more detailed analysis in the appendix.

For the sub-models, after the keys of adapters are set, we individually train M_j by $\mathcal{D}_{a_j}$. Specifically, for $j = 1, \dots, n$, the parameters of M_j is optimized by:

$$\theta_j = \arg \min_{\theta} \sum_{(x, y) \in \mathcal{D}_{a_j}} \mathcal{L}(M_j(\text{En}(x), \theta), y), \quad (4)$$

where $\mathcal{L}$ denotes the loss function. In particular, independent training ensures that the adapter a_j can only be impacted by the samples of $\mathcal{D}_{a_j}$.

Unlearning Process

Unlearning requests are usually intermittent and sequential, e.g., on online services. When a training sample is requested to be unlearned, LegoNet locates and re-trains the impacted adapters to remove the sample's impact. Specifically, for unlearning x , LegoNet just needs, as in the inference process, to encode x as $e = \text{En}(x)$, and then we can find the k nearest adapters to this sample in the encoding space. Assume such k adapters are $a_1^x, a_2^x, \dots, a_k^x$. Because the parameters of the encoder and the keys of the adapters are fixed, the activated adapters are the impacted ones that have been trained on x . During training, we have recorded the set of samples that each adapter has been trained on, i.e., $\mathcal{D}_{a_1}, \dots, \mathcal{D}_{a_n}$. For unlearning, we need to remove the unlearn sample x from $\mathcal{D}_{a_z^x}$ as

$$\mathcal{D}_{a_z^x} \leftarrow \{(x', y') : (x', y') \in \mathcal{D}_{a_z^x}, x' \neq x\} \quad (5)$$

to update the sample set for re-training the sub-model of a_z^x , where $z = 1$ to k . Specifically, the unlearning process of LegoNet is shown in Algorithm 1. Every time LegoNet receives an unlearning request, it will perform the unlearning process once.

Algorithm 1: Unlearning Process of LegoNet

Components: Encoder En, Adapters $\{a_1, \dots, a_n\}$

Sample Records: $\mathcal{D}_{a_1}, \dots, \mathcal{D}_{a_n}$

Unlearning Data: x

Calculate the distance $\sigma(x, a_j)$, for $j = 1, \dots, n$

$\{\mathcal{D}_{a_1^x}, \dots, \mathcal{D}_{a_k^x}\} \leftarrow$ activate k nearest adapters

for $\mathcal{D}_{a_z^x}$ **in** $\{\mathcal{D}_{a_1^x}, \dots, \mathcal{D}_{a_k^x}\}$ **do**

$\mathcal{D}_{a_z^x} \leftarrow \{(x', y') : (x', y') \in \mathcal{D}_{a_z^x}, x' \neq x\}$

$M_{a_z^x} \leftarrow$ Random initial model

Re-train $M_{a_z^x}$ on $\mathcal{D}_{a_z^x}$

end for

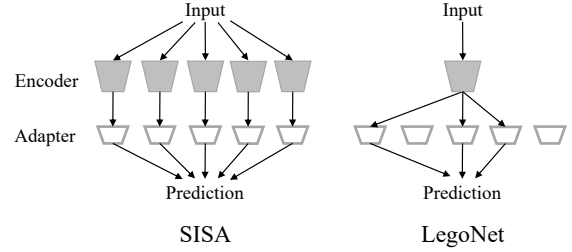


Figure 2: Frameworks of SISA and LegoNet. For SISA, we set the number of shards $s = 5$. For LegoNet, we set $n = 5$ and $k = 3$. In actual use, $k \ll n$ will be satisfied.

Comparing LegoNet with SISA

SISA, as a representative of the previous methods for exact unlearning, also aims to accelerate re-training. In this section, we compare LegoNet with SISA to further discuss the advantages of our method. Following the naming in this paper, in a common framework, the original neural model consists of an encoder and an adapter. On this basic, Fig. 2 shows the frameworks of SISA and LegoNet. SISA splits the training data into s disjoint shards. Then, SISA adopts such s shards to train s models independently. The architecture of each model can be viewed as an encoder connecting an adapter. While LegoNet shares the fixed encoder for n adapters.

Compared with SISA, LegoNet has higher unlearning efficiency. On the one hand, LegoNet has fewer parameters that need to be re-trained. For unlearning a sample, SISA needs to re-train an encoder and an adapter, while LegoNet needs to re-train k adapters. Since k is generally small and the parameters of the encoder are significantly more than that of the adapter. The parameters of the encoder are even much more than that of k adapters.

On the other hand, while maintaining acceptable performance, LegoNet can furthermore reduce the expected number of re-training samples. Assume the training data has N samples. For SISA, each model is expected to train N/s samples. For LegoNet, each adapter is expected to train kN/n samples. Although LegoNet needs to re-train k adapters for unlearning a sample, since $k \ll n$, LegoNet can even achieve $k^2N/n < N/s$. This is thanks to the design of the activation mechanism. The activation mechanism of LegoNet is based on the distance in the encoding space. Similar samples tend to activate the same adapter. When inferring a sample, the knowledge brought from similar samples is usually more

important. Compared to SISA’s randomly assigning samples to train the models, our method makes the learning of the adapter to be more focused on a small region of the encoding space. So our adapters can effectively maintain the performance while requiring fewer samples.

Experiments

In this section, we conduct image classification experiments to present the ability of LegoNet. We first introduce the settings of experiments, including the datasets, baselines, and detailed settings of models. Then we show the performance of our method for various unlearning tasks. Moreover, we analyze the advantages of LegoNet over SISA. Finally, we present the characteristics of LegoNet, through extensive experiments with different hyperparameters.

Experimental settings

Datasets We conduct our experiments on two standard image classification datasets: CIFAR-10 (containing 10 classes) and CIFAR-100 (containing 100 classes). These two datasets are commonly adopted for validating the model performance for classification, and they are also frequently adopted in previous research on machine unlearning (Mahadevan and Mathioudakis 2021; Gupta et al. 2021; Bourtole et al. 2021). In addition, our model requires an external dataset to pre-train the encoder for representation learning. Here, toward the CIFAR datasets, we adopt ImageNet for pre-training the encoder (Bourtole et al. 2021; Kornblith, Shlens, and Le 2019). In particular, our experiments do not discuss the situation where there is no suitable data for pre-training. This question is also the direction of our future works.

Unlearning Tasks To test the unlearning performance, our experiments follow the previous studies (Golatkhar, Achille, and Soatto 2020a; Chundawat et al. 2022). For each dataset, we conduct two kinds of unlearning tasks:

- **Random Sample Unlearning:** we randomly select some training samples to be unlearned from the trained model. In the following, we name the adopted unlearning task to unlearn 100 samples as “**Random100**”.
- **Random Class Unlearning:** we randomly select a class, and the samples in this class are requested to be unlearned. In the following, we name this task as “**UnClass**”.

Performance Evaluation To identify the sample sets, we denote D_{unlearn} as the set of unlearned samples, and denote D_{retain} as the set of the other samples in the training data. Moreover, we denote D_{test} as the test sample set. In the experiments, we will focus on the accuracy on D_{unlearn} , D_{retain} , D_{test} to evaluate the performance of the unlearning strategies.

Implementation Details of Models Since different frameworks may have different strategies for unlearning. Here, we give the setting of different methods in detail. Specifically, in our experiments, there are three frameworks for unlearning:

- **LegoNet:** Our model consists of an encoder and n adapters. Specifically, we adopt a ResNet with 34 layers (ResNet-34 for short, ResNet-34 is deep enough to solve CIFAR-10/100) as our encoder. Each adapter is set as a linear layer.

- **Normal:** The neural network is consist of a ResNet-34 backbone for representation and a linear layer for classification. In particular, we can think of this network as being composed of an encoder and an adapter. In the following, we call this ResNet Classifier as **RNC** for short.

- **SISA:** The model consists s sub-models. Each sub-model adopts the same architecture as the above RNC.

For a fair comparison, ResNet-34s, adopted in LegoNet, RNC, and SISA, are pre-trained on ImageNet. In addition, the unlearning strategies adopted in the experiments are:

- **Re-Train:** Re-training the model on D_{retain} .
- **Tune:** Continue fine-tuning the trained model on D_{retain} (Golatkhar, Achille, and Soatto 2020a).
- **NGrad:** Continue training the trained model on D_{unlearn} with opposite gradients (Golatkhar, Achille, and Soatto 2020a).

Specifically, “Re-Train” is adopted on RNC, SISA, and LegoNet. While “Tune” and “NGrad” are approximate unlearning methods, so we only adopt them on RNC for comparison. For more detailed settings, please refer to the appendix.

Main Results

To demonstrate the performance, we compare the accuracies of our approach on D_{retain} , D_{unlearn} , and D_{test} with multiple unlearning baselines. As shown in Table 2, we can see that: (1) Observing the performance on D_{test} , the accuracy of SISA and LegoNet is lower than that of RNC, which indicates that speeding up unlearning comes at a performance cost. (2) For RNC, re-training is the reliable way to achieve exact unlearning. However, its unlearning costs too much time. Tune and NGrad are approximate unlearning methods. Their unlearning performance is generally judged indirectly by comparing with the accuracy of re-training on D_{retain} and D_{unlearn} . The results of Tune are similar to Re-Train, but the unlearning speed is also too slow because the fine-tuning is processed on D_{retain} . In particular, NGrad unlearns very fast, but it can be found that the unlearning of NGrad is not reliable. In the “UnClass” task for CIFAR-100, the model still holds the probability of inferring the unlearned class, so the impact of this unlearned class is not completely removed. (3) Comparing SISA($s = 10$) and LegoNet($n = 100$), the overall test accuracy of LegoNet is slightly lower than that of SISA (the average is 1.64%). This is the price of the fixed encoder, but in exchange for the faster unlearning speed of LegoNet. As the time cost shown in Fig. 3, LegoNet($n = 100$) only costs 32.65% of the time of SISA($s = 10$) for achieving unlearning. (4) Comparing LegoNet($n = 100$) and LegoNet($n = 1000$), it shows how our method performs when it trades off model performance against unlearning efficiency. Increasing the degree of data segmentation results in higher unlearning speed but leads to a decrease in performance because the samples for training each adapter are reduced. (5) Further comparing LegoNet($n = 1000$) and SISA($s = 10$), The overall test accuracy of LegoNet($n = 1000$) is 2.84% lower than that of SISA($s = 10$). But LegoNet($n = 100$) only costs 12.91% of the time of SISA($s = 10$) for completing the unlearning process. Overall, LegoNet trades a small performance loss for a huge speed boost.

Method \ Task		CIFAR-10						CIFAR-100						Overall Average
		Random100			UnClass			Random100			UnClass			
		D_{retain}	D_{unlearn}	D_{test}	D_{retain}	D_{unlearn}	D_{test}	D_{retain}	D_{unlearn}	D_{test}	D_{retain}	D_{unlearn}	D_{test}	
RNC	Origin	100.0	100.0	95.97	100.0	100.0	95.97	99.98	100.0	80.98	99.98	100.0	80.98	88.48
	Re-train	100.0	96.00	95.85	100.0	0.00	86.41	99.98	81.00	81.23	99.99	0.00	80.33	85.96
	Tune	99.98	97.00	95.88	99.99	0.00	86.03	99.95	90.00	80.71	99.94	0.00	79.71	85.58
	NGrad	96.48	94.00	90.99	89.54	0.00	75.88	92.67	88.00	73.45	95.28	0.50	70.27	77.65
SISA $s = 10$	Origin	93.32	92.00	91.66	92.98	96.35	91.66	81.78	77.00	75.84	81.64	93.75	75.84	83.75
	Re-train	93.14	91.00	91.56	90.56	0.00	79.61	81.09	73.00	74.74	82.25	0.00	75.42	80.33
LegoNet $n = 100$	Origin	93.47	94.00	90.11	93.19	96.03	90.11	87.55	71.00	72.44	87.48	90.00	72.44	81.28
	Re-train	93.46	94.00	89.97	93.49	0.00	80.82	87.58	70.00	72.4	87.46	0.00	71.58	78.69
LegoNet $n = 1000$	Origin	94.66	94.00	89.45	94.47	96.35	89.45	93.59	73.00	70.64	93.49	98.5	70.64	80.05
	Re-train	94.77	95.00	89.45	94.59	0.00	80.31	93.53	77.00	70.46	93.49	0.00	69.75	77.49

Table 2: Accuracy (%) on CIFAR-10/100 for Evaluating the Performance of Unlearning Strategies. The ‘‘Origin’’ rows present the results before unlearning. All LegoNets in this table set $k = 10$. The ‘‘Overall Average’’ presents the average results of all unlearning tasks.

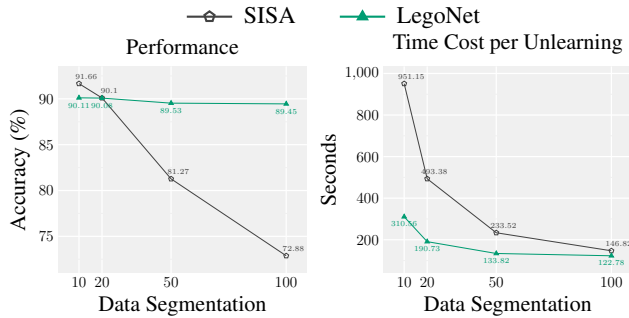


Figure 3: Test Accuracy (%) and Unlearning Time of SISA and LegoNet under Different Degrees of Data Segmentation. Here, the dataset is CIFAR-10. The unlearning task is ‘‘Random100’’. The ‘‘Time Cost per Unlearning’’ shows the average time of unlearning 100 times.

To further demonstrate the advantages of LegoNet in unlearning acceleration, we compare SISA and LegoNet under different degrees of data segmentation on the ‘‘Random100’’ task of CIFAR-10. For SISA, the degree of data segmentation is the value of s . For LegoNet, we set $k = 10$ and the degree of data segmentation is the value of n/k . In particular, when $s = n/k$, the comparison of SISA and LegoNet is relatively fair because the amount of data for re-training each sub-model is expected to be equal. As shown in Fig. 3, we can see that: (1) As the degree of data segmentation increases, the performance of SISA drops dramatically. This indicates that the SISA has a very limited efficiency for unlearning. (2) The unlearning speed of LegoNet is faster than that of SISA when the degree of data segmentation is the same. (3) LegoNet’s performance is well maintained, when n is increased from 100 to 1000. Moreover, the accuracy of LegoNet($n = 1000$) far outperforms that of SISA($s = 100$), which indicates that LegoNet can support a higher degree of data segmentation than SISA. Therefore, LegoNet is more suitable when there are higher rigid requirements for unlearning speed.

The reason why our method can achieve faster unlearning is very intuitive. On the one hand, the number of parameters that LegoNet needs to be re-trained is much smaller than that of SISA. This is because the parameters of the encoder are

thousands of times more than that of an adapter. Specifically, for CIFAR-10, LegoNet($k = 10$) needs to re-train 51.2 thousand parameters. While SISA needs to re-train 3.68 million parameters. On the other hand, due to the simple architecture of the adapter, the number of epochs required for training the adapter is far less than that for training an encoder. Overall, while maintaining acceptable accuracy, LegoNet achieves faster exact unlearning. Therefore, considering the comprehensive ability in terms of speed, accuracy, and unlearning, LegoNet outperforms other baselines.

Ablative Analysis

To study how LegoNet outperforms SISA, i.e., what kinds of differences contribute to the improvements, we ablate them separately. When LegoNet sets $k = 1$, denoted as LegoNet _{$k=1$} , each training sample will only be assigned to one sub-model. Set all sub-models of SISA to share an encoder, and this encoder is fixed during training. We call this model FixSISA. Then, the only difference between FixSISA and LegoNet _{$k=1$} is whether to adopt distance-based data segmentation and sub-model integration. For the above models, we conduct experiments under the condition that the expectation of the number of samples allocated to a single sub-model is equal (i.e., $s = n/k$).

As shown in Table 3, we can see that: (1) The performance of SISA drops seriously when s increases to 50 and 100. This is because the data accepted by each sub-model is too sparse, which greatly increases the learning difficulty. Moreover, compared with SISA, FixSISA performs better when $s = 50, 100$. This indicates that a large number of sub-model parameters makes overfitting more likely to occur, thus affecting the results of the ensemble. (2) Compared with FixSISA, LegoNet _{$k=1$} changes the strategy of assigning data and integrating. From the results, the ability of LegoNet _{$k=1$} is approximately the same as that of FixSISA. But LegoNet _{$k=1$} only adopts one sub-model for inference while FixSISA adopts its all sub-models. It verifies the rationality of our adapter activation mechanism, assigning data based on similarity. The sub-model of LegoNet to infer a sample is trained on the set of similar samples, which means LegoNet only adopts the knowledge of similar samples. While SISA actually uses the knowledge of all the samples. From the comparison be-

Dataset	CIFAR-10				CIFAR-100			
	10	20	50	100	10	20	50	100
SISA	91.66	90.10	81.27	72.88	75.84	72.94	64.20	47.15
FixSISA	88.32	87.00	85.23	83.76	69.39	67.42	64.36	62.44
LegoNet _{k=1}	87.90	87.46	86.40	84.85	67.14	64.85	63.44	61.82
LegoNet _{k=10}	90.11	90.08	89.53	89.45	72.44	72.08	71.10	70.64

Table 3: Accuracy (%) in Ablation Experiments. For SISA and FixSISA, the number in ‘‘Segmentation’’ denotes the value of s . For LegoNet, the number in ‘‘Segmentation’’ denotes the value of n/k .

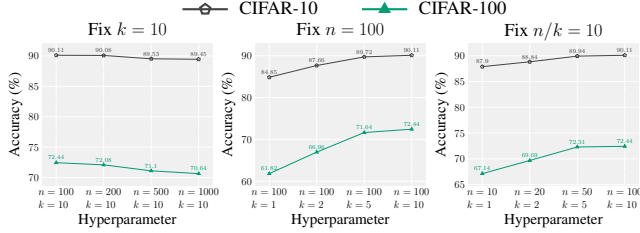


Figure 4: Test Accuracy (%) of LegoNet with Different Hyperparameters.

tween LegoNet_{k=1} and FixSISA, we can see that similar samples are more important for inference. (3) LegoNet has higher freedom in assigning samples because one sample can be assigned to k sub-models. Comparing LegoNet_{k=10} with LegoNet_{k=1}, it shows that the overall capability of LegoNet is greatly improved when k increases.

LegoNet with Different Hyperparameters

To show the characteristics of LegoNet, we evaluate LegoNet’s ability with different hyperparameters. First, we discuss the results of LegoNet by fixing k , n , and n/k , respectively. As shown in Fig. 4, we can see that: (1) When fixing $k=10$, the increase of n will lead to a decrease in model performance, because the data received by each sub-model is significantly reduced. However, LegoNet maintains performance well. Comparing $n=100$ and $n=1000$ the accuracy only decreases by 0.66% in CIFAR-10 and 1.80% in CIFAR-100. (2) When fixing $n=100$, the increase of k will lead to increasing the model performance. There are two reasons for this. On the one hand, the samples for training each sub-model are increased. On the other hand, more sub-models have joined the ensemble of results. (3) When fixing $n/k=10$, LegoNet’s performance improves as n and k are increased synchronously. The expected number of samples received by each sub-model is unchanged. So the performance improvement here is brought about by integrating more sub-models. Overall, the performance of LegoNet is stable when k is large enough. In practical tasks, a large increase on n can greatly improve the unlearning speed of LegoNet with a small performance loss. When performance needs to be increased, we can try to increase k . Based on the adjustment of n and k , LegoNet can intuitively trade off the unlearning speed against the model performance.

Moreover, we also present the count of parameters and FLOPs of LegoNet with different hyperparameters. To intuitively show our advantages, we still choose to compare with SISA. As shown in Fig. 5, we can see that: (1) When increasing n , the parameters of LegoNet change little. Be-

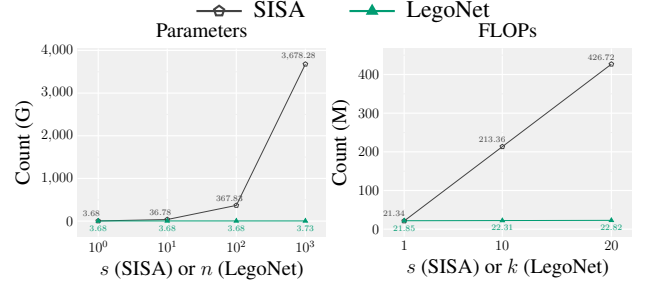


Figure 5: Count of parameters and FLOPs for SISA and LegoNet. In ‘‘FLOPs’’, LegoNet sets $n=1000$. The FLOPs of LegoNet contain the computation of network inference and adapter activation.

cause the number of parameters that an adapter has is usually small. So LegoNet requires less memory space. (2) When increasing k , the FLOPs of LegoNet change little. Because the encoder that needs larger FLOPs only requires to infer once. Furthermore, LegoNet generally sets a small k . (3) The FLOPs of our method are significantly lower than those of SISA, which indicates the inference of LegoNet is faster.

The advantages in the number of parameters and the amount of computation allow our method to be more easily deployed in practical applications. For example, the providers of online services can switch to our model at a low cost, without the need to upgrade equipment.

Conclusion

In this paper, we present a novel network architecture, namely *LegoNet* to fast and completely erase the impact of specific training samples from a deep neural network. Compared with the original model, our method only adds a very small number of additional parameters and inference calculations. Moreover, our method has great potential. When pre-training methods are improved, our method will also benefit and achieve better performance. From a research perspective, our method provides a new idea to speed up unlearning by both reducing the number of parameters and samples involved in the re-training process. According to this idea, the specific implementation scheme of our proposed method still has a large space for improvement. For example, the improvement of the encoder, a better activation mechanism for the adapters, etc. Furthermore, because LegoNet has a strong ability to control the impact of samples, in addition to re-training, there may be more clever strategies to achieve unlearning. We believe these future studies will be interesting and have important implications for society.

References

- Alessandro; and Mantelero. 2013. The EU Proposal for a General Data Protection Regulation and the roots of the 'right to be forgotten'. *Computer Law & Security Report*.
- Aljundi, R.; Chakravarty, P.; and Tuytelaars, T. 2017. Expert gate: Lifelong learning with a network of experts. In *Proceedings of CVPR*, 3366–3375.
- Bourtole, L.; Chandrasekaran, V.; Choquette-Choo, C. A.; Jia, H.; Travers, A.; Zhang, B.; Lie, D.; and Papernot, N. 2021. Machine unlearning. In *Proceedings of S&P*, 141–159. IEEE.
- Cao, Y.; and Yang, J. 2015. Towards making systems forget with machine unlearning. In *Proceedings of S&P*, 463–480. IEEE.
- Carlini, N.; Liu, C.; Erlingsson, U.; Kos, J.; and Song, D. 2019. The Secret Sharer: Evaluating and Testing Unintended Memorization in Neural Networks. In *Proceedings of USENIX Security*, 267–284. USA: USENIX Association. ISBN 9781939133069.
- Carlini, N.; Tramèr, F.; Wallace, E.; Jagielski, M.; Herbert-Voss, A.; Lee, K.; Roberts, A.; Brown, T.; Song, D.; Erlingsson, Ú.; Oprea, A.; and Raffel, C. 2021. Extracting Training Data from Large Language Models. In *Proceedings of USENIX Security*, 2633–2650. USENIX Association. ISBN 978-1-939133-24-3.
- Cauwenberghs, G.; and Poggio, T. 2001. Incremental and Decremental Support Vector Machine Learning. In *Advances in NIPS*, volume 13, 388–394. MIT Press.
- Chen, C.; Sun, F.; Zhang, M.; and Ding, B. 2022. Recommendation Unlearning. In *Proceedings of WWW*, 2768–2777. New York, NY, USA: Association for Computing Machinery. ISBN 9781450390965.
- Chen, M.; Zhang, Z.; Wang, T.; Backes, M.; Humbert, M.; and Zhang, Y. 2021. Graph Unlearning. *arXiv preprint arXiv:2103.14991*.
- Chundawat, V. S.; Tarun, A. K.; Mandal, M.; and Kankanhalli, M. 2022. Zero-Shot Machine Unlearning. *arXiv preprint arXiv:2201.05629*.
- Ginart, A.; Guan, M.; Valiant, G.; and Zou, J. Y. 2019. Making AI Forget You: Data Deletion in Machine Learning. In *Advances in NIPS*, volume 32. Curran Associates, Inc.
- Golatkhar, A.; Achille, A.; and Soatto, S. 2020a. Eternal sunshine of the spotless net: Selective forgetting in deep networks. In *Proceedings of CVPR*, 9304–9312.
- Golatkhar, A.; Achille, A.; and Soatto, S. 2020b. Forgetting Outside the Box: Scrubbing Deep Networks of Information Accessible from Input-Output Observations. In *Proceedings of ECCV*, 383–398. Cham. ISBN 978-3-030-58526-6.
- Guo, C.; Goldstein, T.; Hannun, A.; and Van Der Maaten, L. 2020. Certified Data Removal from Machine Learning Models. In III, H. D.; and Singh, A., eds., *Proceedings of ICML*, volume 119, 3832–3842. PMLR.
- Gupta, V.; Jung, C.; Neel, S.; Roth, A.; Sharifi-Malvajerdi, S.; and Waites, C. 2021. Adaptive machine unlearning. *Advances in NIPS*, 34: 16319–16330.
- He, K.; Zhang, X.; Ren, S.; and Sun, J. 2016. Identity mappings in deep residual networks. In *Proceedings of ECCV*, 630–645. Springer.
- Izzo, Z.; Smart, M. A.; Chaudhuri, K.; and Zou, J. 2021. Approximate Data Deletion from Machine Learning Models. In *Proceedings of AISTATS*, volume 130, 2008–2016. PMLR.
- Karasuyama, M.; and Takeuchi, I. 2009. Multiple Incremental Decremental Learning of Support Vector Machines. In *Advances in NIPS*, volume 22. Curran Associates, Inc.
- Kornblith, S.; Shlens, J.; and Le, Q. V. 2019. Do better imagenet models transfer better? In *Proceedings of CVPR*, 2661–2671.
- Mahadevan, A.; and Mathioudakis, M. 2021. Certifiable machine unlearning for linear models. *arXiv preprint arXiv:2106.15093*.
- Neel, S.; Roth, A.; and Sharifi-Malvajerdi, S. 2021. Descent-to-Delete: Gradient-Based Methods for Machine Unlearning. In *Proceedings of ALT*, volume 132, 931–962. PMLR.
- Russakovsky, O.; Deng, J.; Su, H.; Krause, J.; Satheesh, S.; Ma, S.; Huang, Z.; Karpathy, A.; Khosla, A.; and Bernstein, M. 2015. ImageNet Large Scale Visual Recognition Challenge. *Int J Comput Vis*, 115(3): 211–252.
- Schelter, S. 2020. "Amnesia" - Machine Learning Models That Can Forget User Data Very Fast. In *Workshop on AIDB at VLDB*.
- Shapiro, A. 2003. Monte Carlo sampling methods. *Handbooks in operations research and management science*, 10: 353–425.
- Shokri, R.; Stronati, M.; Song, C.; and Shmatikov, V. 2017. Membership inference attacks against machine learning models. In *Proceedings of S&P*, 3–18. IEEE.
- Torralba, A.; Fergus, R.; and Freeman, W. T. 2008. 80 Million Tiny Images: A Large Data Set for Nonparametric Object and Scene Recognition. *IEEE Trans. Pattern Anal. Mach. Intell.*, 30(11): 1958–1970.
- Truex, S.; Liu, L.; Gursoy, M. E.; Yu, L.; and Wei, W. 2019. Demystifying membership inference attacks in machine learning as a service. *IEEE Trans. Serv. Comput.*
- Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A. N.; Kaiser, Ł.; and Polosukhin, I. 2017. Attention is all you need. *Advances in NIPS*, 30.
- Wang, R.; Tang, D.; Duan, N.; Wei, Z.; and Zhou, M. 2021. K-Adapter: Infusing Knowledge into Pre-Trained Models with Adapters. In *Proceedings of ACL-IJCNLP*.
- Yeom, S.; Giacomelli, I.; Fredrikson, M.; and Jha, S. 2018. Privacy risk in machine learning: Analyzing the connection to overfitting. In *Proceedings of CSF*, 268–282. IEEE.
- Zanella-Béguelin, S.; Wutschitz, L.; Tople, S.; Rühle, V.; Paverd, A.; Ohrimenko, O.; Köpf, B.; and Brockschmidt, M. 2020. Analyzing Information Leakage of Updates to Natural Language Models. In *Proceedings of CCS*, 363–375.
- Zhang, M.; Ren, Z.; Wang, Z.; Ren, P.; Chen, Z.; Hu, P.; and Zhang, Y. 2021. Membership Inference Attacks Against Recommender Systems. In *Proceedings of CCS*, 864–879. New York, NY, USA: Association for Computing Machinery. ISBN 9781450384544.